

JavaScript: The New Parts

JavaScript continues to grow from strength to strength because of its versatility. But change is the law of life and ES6 has given way to ES7 in 2016. This article is the seventh in the series, and talks of all that is new in ES7.

In the last six articles in this series, we learned about the different features of the JavaScript version that is officially called ECMAScript 2015. It is also called ES6, for short. This is by far the biggest version after the first version of the language, in terms of the number of features packed in a single release. It took around six years for this version to become a standard, though the seeds of some features were sown a decade back. When the Web is transforming so fast, six years is an era. Many browser vendors were unhappy with such a slow process of standardisation, and so the technical committee (TC39) decided to do annual, time-bound releases. The promise has been kept with the release of ES7 or the ECMAScript 2016 version, which was announced at the end of January 2016.

The committee and the process

As part of the ECMA organisation, the Technical Committee 39 (TC39) is chartered to define language specifications. All development happens in an open and transparent way. This is not a trivial matter. Many popular languages that were backed by a company have had their priorities changed when the company got acquired. Since the time ECMA took over, developers, browser vendors and companies have participated in the standardisation process. All the minutes of the meetings and progress status reports are updated at github.com/tc39 link, which is a very friendly and familiar interface for developers. You can give inputs on your favourite feature by just submitting an 'issue' or participating in the conversation on GitHub. It would be great if JavaScript fans in India participated in defining the language.

To understand the progress made in 2015 (for ES7), we should first understand the phases which every feature passed through before becoming standard (see Table 1).

No	Stage	Purpose
0	Strawman	Collect the feature being proposed
1	Proposal	Identify value, solution, potential challenges
2	Draft	Describe syntax, semantics and formal spec language
3	Candidate	Refine based on implementations and users
4	Finished	Ready for inclusion in the upcoming version

Table 1: Feature maturity stages

There are only two new features in ES7 -- *Array.prototype.includes()* and *Exponentiation operator*. Compared to the size of ES6, the next version is very tiny. But, that's not all. Many new features have also progressed to the definition stage during 2015.

Array.prototype.includes()

This method returns a Boolean based on whether an element is present in the array or not -- true if present, false if not. Finding an element could be achieved even before this method, by using *indexOf()*.

The command used before ES7 was released is:

```
[ 'ES5', 'ES6', 'ES7' ].indexOf('ES6') > -1; // true
```

The command used after ES7 has been released is:

```
[ 'ES5', 'ES6', 'ES7' ].includes('ES6'); // true
```

Like many other features *includes()* improves readability.

Exponentiation operator **

Before the release of ES7, we used the *Math.pow()* operator for the exponentiation operation. Now a new operator ****** has been introduced:

```
js> 2**3
8
```

Features at the 'candidate' stage

There could be some amount of disappointment with the ES7 version. But there is much progress on the big features. Table 2 gives the features that have reached the 'candidate' stage. These features will more likely make it into the next version, and are being refined based on feedback from developers and browser vendors.

SIMD

SIMD stands for Single Instruction Multiple Data. Before understanding it, let's first look at Single Instruction Single Data (SISD).

SISD is adding two scalars (individual values) in a single operation. To add four pairs of values, you need to perform the operation four times. In case of SIMD, you operate on vectors (array) of four values each. You can add two sets of four values in a single operation. From